Real-Time Chat Thread

A system-design and architecture analysis for a single native Android feature — treated like a mini design interview before a line of code is written.

LAYER 01

Presentation

Compose UI, thin ViewModel, unidirectional state & effects.

LAYER 02

Domain

Pure-Kotlin UseCases. Zero Android deps. Fully unit-testable.

LAYER 03

Data

Room as single source of truth · fakeable streaming source.

PLATFORM

Native Android — Kotlin + Jetpack Compose

ARCHITECTURE

UseCase-heavy Clean Architecture + MVVM

CONCURRENCY

Kotlin Coroutines + Flow · Hilt DI

FOCUS

Optimistic UI · stable ordering · testable boundaries · lifecycle correctness

Before writing any code, I treated this like a mini system-design interview. What follows are the **requirements I extracted**, the **assumptions I made**, the **questions I asked myself**, and the **key decisions** I reached — with the architecture and data flows worked out up front so implementation becomes mechanical.

01 Functional Requirements

What the screen must do.

Single chat thread
Display one focused conversation view.

Inbound streaming
Messages arrive from a streaming source while the user is on screen.

Typing indicator
Reflects live activity from the stream.

Optimistic send
Message appears instantly as `PENDING` → `SENT` on success → `FAILED` with retry on error.

Stable ordering, always
No duplicates, no lost or skipped messages, correct sequence even under rapid inbound traffic.

Unidirectional state
State is predictable and flows one way.

Pure, testable logic
Core business logic has no UI or real-stream dependencies.

Interfaced stream
Streaming source sits behind an interface — fakeable for tests & a future real WebSocket.

Lifecycle correctness across

- Configuration changes (rotation)
- Process death and recreation
- Background / foreground transitions

02 Out of Scope

Deliberately excluded to keep the exercise focused on the hard parts.

- × Full chat list / multi-chat navigation
- × User authentication or profiles
- × End-to-end encryption or message security
- × Delivery / read receipts beyond sent & failed
- × Media, voice messages, or rich content
- × Full history pagination with `RemoteMediator`
- × A real backend or WebSocket — a controllable fake stream stands in
- × Offline sync with `WorkManager` — basic version only if time permits

03 Assumptions

- We are building **one focused screen**, not a full messaging app.
- A real backend exists in production; here we **simulate the streaming source** so we can control rapid traffic and typing events for demonstration and testing.
- Messages carry both a **client timestamp** (immediate optimistic ordering) and a **server timestamp** (on successful send).
- `Room` is acceptable and desirable as the **Single Source of Truth** — reactivity and persistence for free.
- "Rapid inbound traffic" means we can **simulate bursts** of messages and prove ordering stays stable.

04 Architecture & Key Decisions

The questions I asked myself before writing code, and the decisions they led to.

Q1 Which architecture — and why?

MVVM + UseCase-heavy Clean Architecture. The feature has several non-trivial requirements: optimistic UI, rapid inbound messages without duplicates or ordering issues, a fakeable stream, and highly testable business logic.

Core business rules — creating a pending message, sending it, handling success / failure, reconciling with server responses — live in dedicated UseCases (chiefly `SendMessageUseCase`). This keeps the logic **pure Kotlin**, independent of Android, Room, and the UI, so it is trivial to unit-test with `runTest` + `Turbine` — essential for proving stable ordering under burst traffic.

The ViewModel stays thin: it orchestrates and exposes `StateFlow<ChatUiState>` and `SharedFlow<ChatEffect>`. All data access goes through a Repository backed by `Room` as the single source of truth — automatic reactivity via Flows and stable ordering via `ORDER BY timestamp`.

I considered **plain MVVM** (pushes logic into the ViewModel — harder to test/reuse) and **MVI** (excellent for complex state, but more boilerplate than this needs). Clean + UseCases gave the best balance of testability, clarity, and maintainability — and the production-grade thinking that scales to 40+ screens.

Q2 How is stable ordering guaranteed under rapid traffic + optimistic sends?

Every message gets a **client-generated timestamp** (immediate ordering) plus a unique **UUID** (for optimistic messages). The Room query uses `ORDER BY timestamp ASC`. On server success we **update the same row** rather than inserting a duplicate, via an `INSERT OR REPLACE` strategy.

Q3 How do messages stay correctly ordered when sends and inbound arrive at once? Server timestamps only?

No — sorting depends on **who sent the message**:

Your own messages

Always prefer `clientTimestamp` — even after server confirmation. The message never jumps position once it appears.

Messages from others

Use `serverTimestamp` — the authoritative source of truth for remote ordering.

Q4 How is the ViewModel kept thin and testable?

It only orchestrates. It exposes a `StateFlow<ChatUiState>`, a `SharedFlow<ChatEffect>`, and a `Flow<PagingData<Message>>` (or `Flow<List<Message>>`). All business logic lives in the UseCases; one-time effects use the `SharedFlow`.

Q5 How is lifecycle handled without leaks?

`ViewModel` + `viewModelScope`, with `collectAsStateWithLifecycle()` in Compose. Room Flows are lifecycle-aware. The fake stream is started and stopped from the ViewModel — or a lifecycle observer if needed.

Q6 How is “no duplicates or skips” demonstrated under rapid traffic?

A **“Simulate Rapid Inbound”** button in the debug UI fires 20–50 messages quickly. The unit tests plus visible behaviour in the running app together prove correct ordering and zero duplicates.

Q7 Why Coroutines + Flow over RxJava, LiveData, or raw callbacks?

Coroutines give structured concurrency that pairs naturally with `viewModelScope` and Compose. Flow is cold, backpressure-aware, and first-class in Room, Paging 3, and Retrofit — making tests deterministic with `runTest` + `Turbine`. Google’s entire modern Android stack is built on Coroutines, so anything else adds needless complexity.

Q8 Why Hilt for dependency injection?

Hilt is Google’s official DI for Android. It removes large amounts of Dagger boilerplate, integrates cleanly with ViewModel, WorkManager, and Compose, and makes test fakes trivial via `@TestInstallIn`. For a codebase growing to 40+ screens, the consistency outweighs the small learning curve. We inject UseCases into ViewModels, Repositories into UseCases, DAOs into Repositories, and the fake stream into the data layer.

05 Architecture Overview

Three layers, one dependency direction. Arrows point inward — outer layers depend on inner abstractions, never the reverse.

Presentation · Domain (pure Kotlin) · Data · External / source

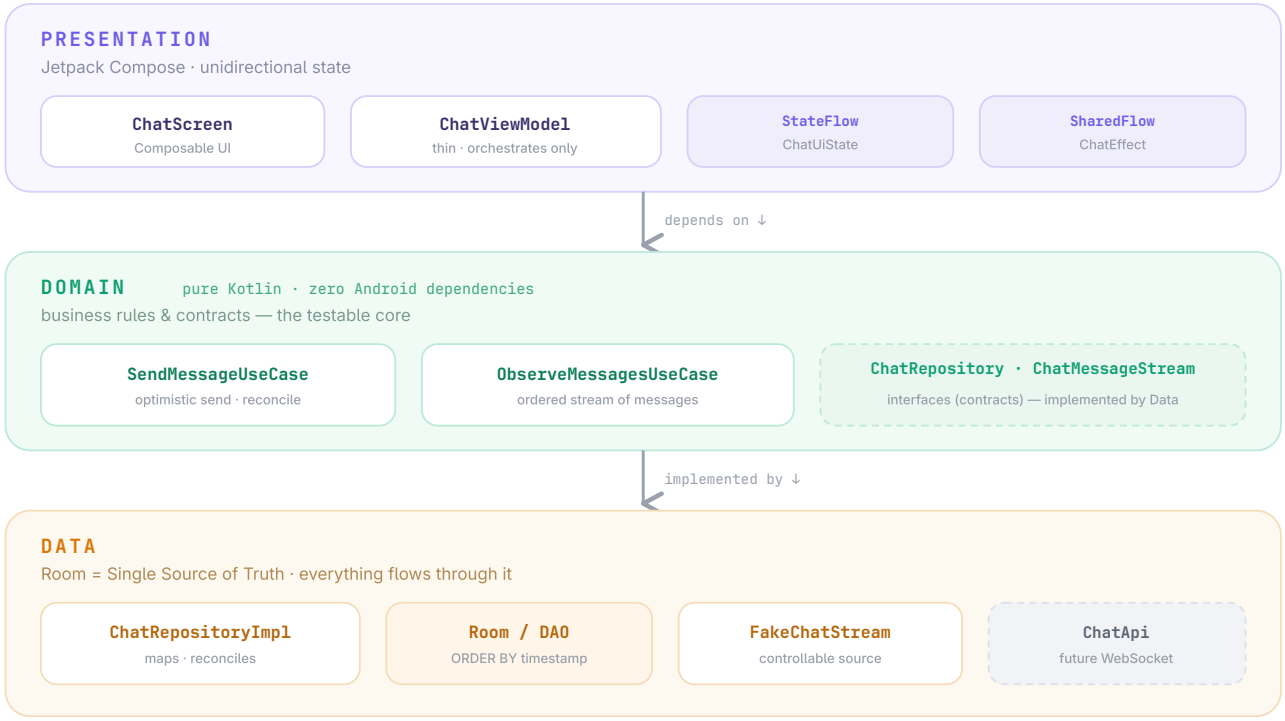

FIGURE 1 · CLEAN ARCHITECTURE LAYERS AND INWARD DEPENDENCY DIRECTION

06 Data & State Flows

Optimistic send

1. User taps **Send** → `ChatScreen` → `viewModel.onAction(SendMessage(text))`.
2. `ChatViewModel` → `SendMessageUseCase(chatId, text)`.
3. UseCase creates a `Message` with status `PENDING`, a client timestamp, and a temp UUID.
4. `repository.insertPendingMessage()` → Room insert. The Room Flow emits, so the UI shows the message **instantly**.
5. `repository.sendMessageToServer()` is called.
6. **Success** → `updateMessageStatus(id, Sent(serverTimestamp))` → UI shows a checkmark.
7. **Failure** → `updateMessageStatus(id, Failed(reason))` → UI shows a retry button.

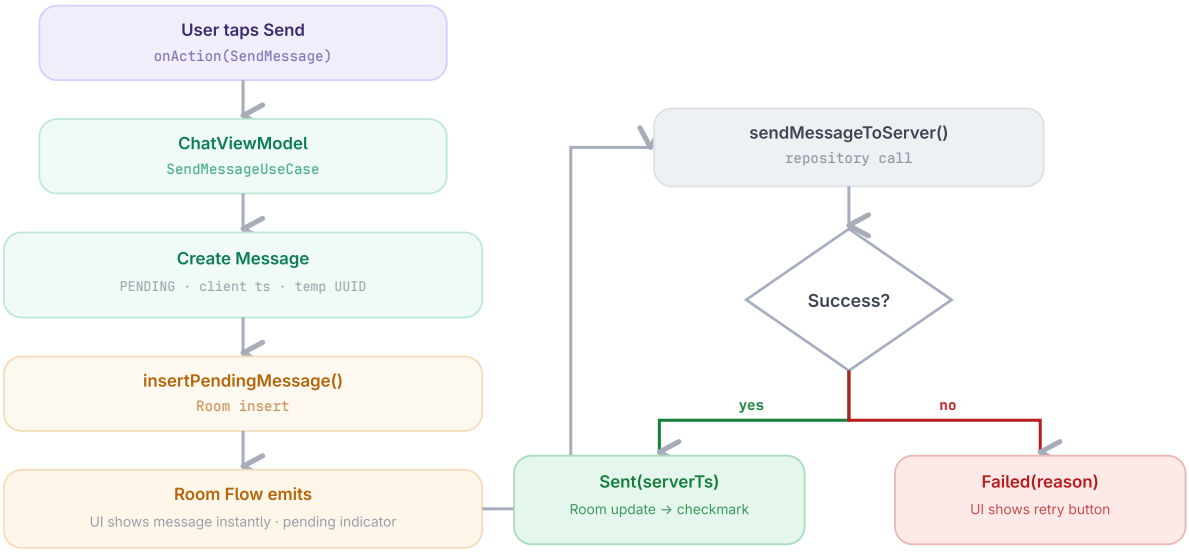

FIGURE 2 · OPTIMISTIC SEND — INSTANT LOCAL INSERT, THEN RECONCILE WITH THE SERVER RESULT

Real-time inbound

The `FakeChatStream` (or a real WebSocket) emits a message → `ChatRepositoryImpl` parses and maps it → `chatDao.insertOrUpdate()` → the Room change makes `ObserveMessagesUseCase` emit a new `PagingData` (or List) → Compose updates automatically.

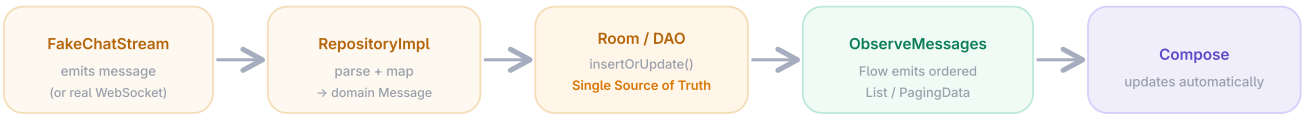

FIGURE 3 · INBOUND PIPELINE — EVERY MESSAGE FLOWS THROUGH ROOM BEFORE IT EVER REACHES THE UI

The ViewModel and UI never listen to the stream directly. Everything flows through Room — that single funnel is what makes ordering, de-duplication, and lifecycle behaviour predictable.

Row-level reconciliation on send

On server acknowledgement we update the existing row — never insert a second one. The same primary key carries the message from pending to sent.

What changes in the database row

`UPDATE — same row, no duplicate insert`

COLUMN	AFTER SEND (OPTIMISTIC)	AFTER SERVER ACK	CHANGE
id	`temp1`	`serverId123`	Updated
serverId	`null`	`serverId123`	Filled in
content	“Hey, how are you?”	“Hey, how are you?”	No change
timestamp	1000000001	1000000001	Usually unchanged
status	`PENDING`	`SENT`	Updated
senderId	user_prakash_123	user_prakash_123	No change

07 Testing Strategy

Testing concentrates on the **domain layer**, because the business logic is pure and deterministic. ViewModel tests stay light — it only orchestrates.

`SendMessageUseCase` `HIGHEST`
Optimistic send and full status transition sequence.

Optimistic creation
Correct client timestamp and `PENDING` status on creation.

Success path
Message updates to `SENT` with server timestamp in place.

Failure path
Message updates to `FAILED` with retry available.

Ordering logic
Stable ordering proven under rapid inbound traffic.

Conflict resolution
Inbound message with same tempId must not create a duplicate.

Duplicate prevention
Stream echoing an optimistic message is de-duplicated.

`FakeChatStream`
Proven fully controllable for tests and the debug demo.

Reconciliation
Success & failure verified with `runTest` + `Turbine`.

`ObserveMessagesUseCase`
Room Flow emits the correctly ordered list.

Edge cases covered: offline send, message burst, and failure + retry. **Overall goal —** prove stable ordering, optimistic reconciliation, and a fakeable stream with clean unit tests.

08 Project Structure

Clean + feature-based. Colours match the architecture layers above.

presentation domain data

```
app/
├── core/
│   ├── database/
│   ├── di/
│   └── util/
├── feature/chat/
│   ├── data/
│   │   ├── local/
│   │   ├── remote/            # ChatApi placeholder
│   │   ├── stream/            # FakeChatStream + interface
│   │   ├── repository/
│   │   └── mapper/
│   ├── domain/
│   │   ├── model/
│   │   ├── repository/        # interfaces only
│   │   └── usecase/
│   └── presentation/
│       ├── chat/
│       │   ├── ChatScreen.kt
│       │   ├── ChatViewModel.kt
│       │   ├── ChatUiState.kt
│       │   ├── ChatAction.kt
│       │   └── ChatEffect.kt
│       └── components/
├── MainActivity.kt
└── navigation/
```

✓ **The domain layer has zero Android dependencies** — pure Kotlin, perfect for fast, deterministic unit tests.